Project 1 -
Unit Testing
CppUnit & Gcovr

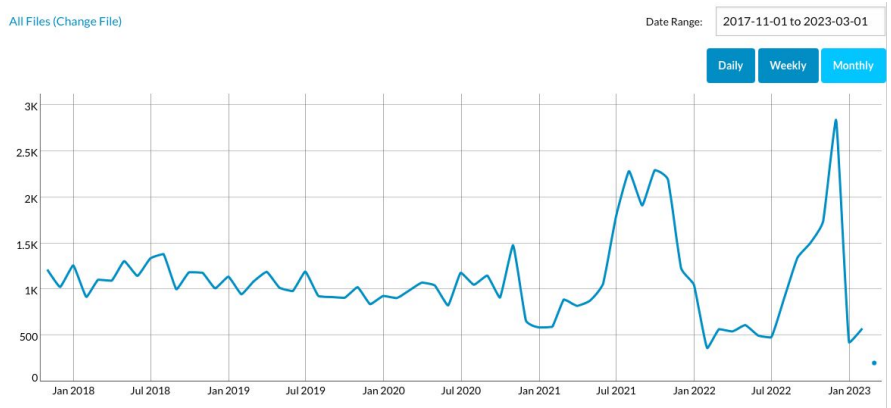
TA:劉宗翰
Email:r11921094@ntu.edu.tw
Software Testing and Security Checking
Department of EE, National Taiwan University

Unit Test

- White-box testing
- For the quality assurance(QA) of a unit part of a program
- A must step before submitting a unit to a big project

CppUnit

- The C++ port of the famous JUnit framework for unit testing
- Started around 2000 by Michael Feathers
- A defacto standard of C++ unit testing package
- https://cppunit.sourceforge.net/doc/cvs/index.html



Gcovr

- Generate GCC code coverage reports
- Gcovr provides a utility for managing the use of the GNU gcov utility and generating summarized code coverage results
  - use special compiler options to compile enables gcovr to analyze
- https://gcovr.com/en/stable/index.html

Implementing Test Framework with CppUnit

1. Design your test cases in **TestFixture**
    a. test under specific scenario whether it has expected result
2. Add test cases to your **TestSuite**
    a. usually combine related test cases into one test suite
3. Regist **TestSuite** to **TestFactoryRegistry**
4. Add **TestSuite** from **TestFactoryRegistry** to **TestRunner**
5. Invoke **TestRunner::run()** to run the testing
6. Compile and Run

TestFixture

- Used to provide a common environment for a set of test cases

Public Member Functions

virtual	**~TestFixture** ()
virtual void	**setUp** () *Set up context before running a test.*
virtual void	**tearDown** () *Clean up after the test run.*

```
class ComplexNumberTest : public CppUnit::TestFixture {
private:
  Complex *m_10_1, *m_1_1, *m_11_2;
public:
  void setUp()
  {
    m_10_1 = new Complex( 10, 1 );
    m_1_1 = new Complex( 1, 1 );
    m_11_2 = new Complex( 11, 2 );
  }

  void tearDown()
  {
    delete m_10_1;
    delete m_1_1;
    delete m_11_2;
  }
};
```

Test Cases

- How to write and invoke individual tests using a fixture?
  - Write the test cases as a method in the fixture class
  - Create a **TestCaller** which runs that particular method

```
class ComplexNumberTest : public CppUnit::TestFixture  {
private:
  Complex *m_10_1, *m_1_1, *m_11_2;
public:
  void testEquality()
  {
    CPPUNIT_ASSERT( *m_10_1 == *m_10_1 );
    CPPUNIT_ASSERT( !(*m_10_1 == *m_11_2) );
  }

  void testAddition()
  {
    CPPUNIT_ASSERT( *m_10_1 + *m_1_1 == *m_11_2 );
  }
};
```

- Test functions in **TestFixture**

Test Cases

- How to write and invoke individual tests using a fixture?
  - Write the test cases as a method in the fixture class
  - Create a **TestCaller** which runs that particular method

```
CppUnit::TestCaller<ComplexNumberTest> test( "testEquality",
                                             &ComplexNumberTest::testEquality );
CppUnit::TestResult result;
test.run( &result );
```

❏ **TestCaller** in main function

Assertions

- Verification statement as a function call to CppUnit
- CppUnit also provides some useful assertions
  - https://cppunit.sourceforge.net/doc/cvs/group__assertions.html

#define	**CPPUNIT_ASSERT**(condition) *Assertions that a condition is* `true`.
#define	**CPPUNIT_ASSERT_MESSAGE**(message, condition) *Assertion with a user specified message.*
#define	**CPPUNIT_FAIL**(message) *Fails with the specified message.*
#define	**CPPUNIT_ASSERT_EQUAL**(expected, actual) *Asserts that two values are equals.*
#define	**CPPUNIT_ASSERT_EQUAL_MESSAGE**(message, expected, actual) *Asserts that two values are equals, provides additional message on failure.*
#define	**CPPUNIT_ASSERT_DOUBLES_EQUAL**(expected, actual, delta) *Macro for primitive double value comparisons.*
#define	**CPPUNIT_ASSERT_DOUBLES_EQUAL_MESSAGE**(message, expected, actual, delta) *Macro for primitive double value comparisons, setting a user-supplied message in case of failure.*

TestSuite

- Set up tests so that you can run them all at once
- Combine together into one **TestSuite**

```
CppUnit::TestSuite suite;
CppUnit::TestResult result;
suite.addTest( new CppUnit::TestCaller<ComplexNumberTest>(
                    "testEquality",
                    &ComplexNumberTest::testEquality ) );
suite.addTest( new CppUnit::TestCaller<ComplexNumberTest>(
                    "testAddition",
                    &ComplexNumberTest::testAddition ) );
suite.run( &result );
```

❏ Add test cases to TestSuite

```
CppUnit::TestSuite suite;
CppUnit::TestResult result;
suite.addTest( ComplexNumberTest::suite() );
suite.addTest( SurrealNumberTest::suite() );
suite.run( &result );
```

❏ Combine multiple suites into one suite

TestRunner

- Run suite and collect the results

```
#include <cppunit/ui/text/TestRunner.h>
#include "ExampleTestCase.h"
#include "ComplexNumberTest.h"

int main( int argc, char **argv)
{
  CppUnit::TextUi::TestRunner runner;
  runner.addTest( ExampleTestCase::suite() );
  runner.addTest( ComplexNumberTest::suite() );
  runner.run();
  return 0;
}
```

❏ Add two suites to **TestRunner** and run

```
public:
  static CppUnit::Test *suite()
  {
    CppUnit::TestSuite *suiteOfTests = new CppUnit::TestSuite( "ComplexNumberTest" );
    suiteOfTests->addTest( new CppUnit::TestCaller<ComplexNumberTest>(
                                  "testEquality",
                                  &ComplexNumberTest::testEquality ) );
    suiteOfTests->addTest( new CppUnit::TestCaller<ComplexNumberTest>(
                                  "testAddition",
                                  &ComplexNumberTest::testAddition ) );
    return suiteOfTests;
  }
```

❏ Implement **TestSuite** in the test class

Helper Macros

- Shorten the code

```
CPPUNIT_TEST_SUITE( ComplexNumberTest );
CPPUNIT_TEST( testEquality );
CPPUNIT_TEST( testAddition );
CPPUNIT_TEST_SUITE_END();
```

- ❏ An test suite implementation equivalent to below

```
public:
  static CppUnit::Test *suite()
  {
    CppUnit::TestSuite *suiteOfTests = new CppUnit::TestSuite( "ComplexNumberTest" );
    suiteOfTests->addTest( new CppUnit::TestCaller<ComplexNumberTest>(
                                  "testEquality",
                                  &ComplexNumberTest::testEquality ) );
    suiteOfTests->addTest( new CppUnit::TestCaller<ComplexNumberTest>(
                                  "testAddition",
                                  &ComplexNumberTest::testAddition ) );
    return suiteOfTests;
  }
```

- ❏ Suites implementation in the test class

TestFactoryRegistry

```
#include <cppunit/extensions/HelperMacros.h>

CPPUNIT_TEST_SUITE_REGISTRATION( ComplexNumberTest );
```

- Store fixture suite
- Register fixture suites at initialization time
- Created to solve two pitfalls
  - forget to add your fixture suite to the test runner
  - compilation bottleneck caused by the inclusion of all test case headers

❏ Regist test suite in test class

```
#include <cppunit/extensions/TestFactoryRegistry.h>
#include <cppunit/ui/text/TestRunner.h>

int main( int argc, char **argv)
{
  CppUnit::TextUi::TestRunner runner;

  CppUnit::TestFactoryRegistry &registry = CppUnit::TestFactoryRegistry::getRegistry()

  runner.addTest( registry.makeTest() );
  runner.run();
  return 0;
}
```

❏ Retrieve suite from **TestFactoryRegistry**

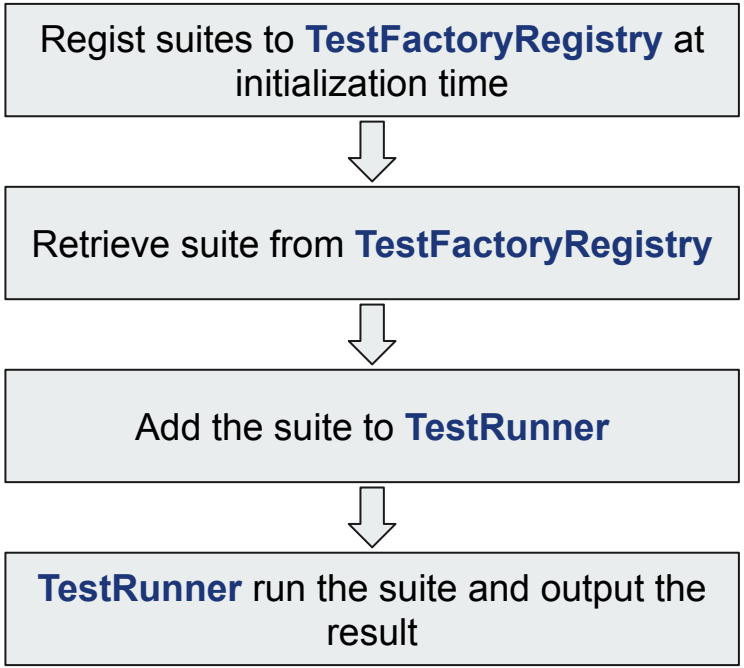
Installation

- Ubuntu 20.04
  - this tutorial was written and tested on Ubuntu 20.04
- For CppUnit
  - sudo apt-get install libcppunit-dev
- For gcovr (line coverage tool)
  - sudo apt-get install gcovr

Example Project

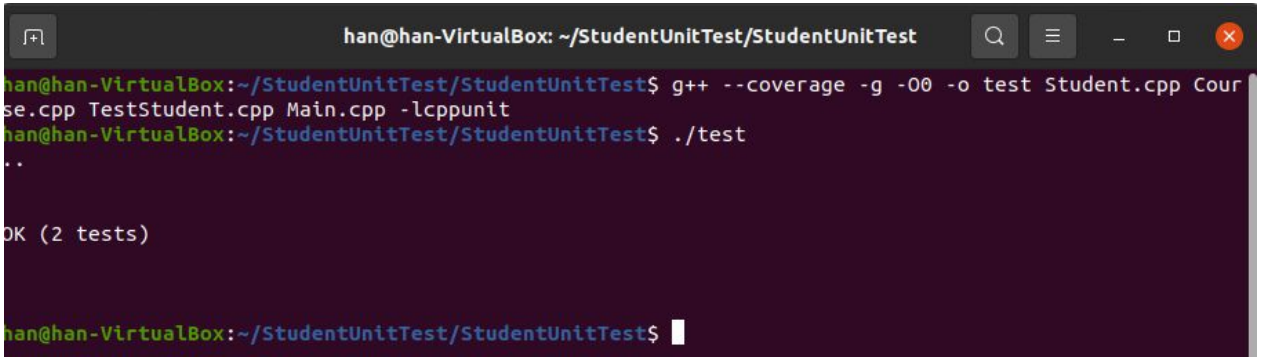
- Refer to NUS: Crash Course in using CppUnit
- Program under test
  - Student
    - 記錄修過的課程(Course)
  - Course
    - 記錄課程名稱及成績
- Test cases
  - Student的constructor
  - Student成績的assign和retrieve

Execution Flow



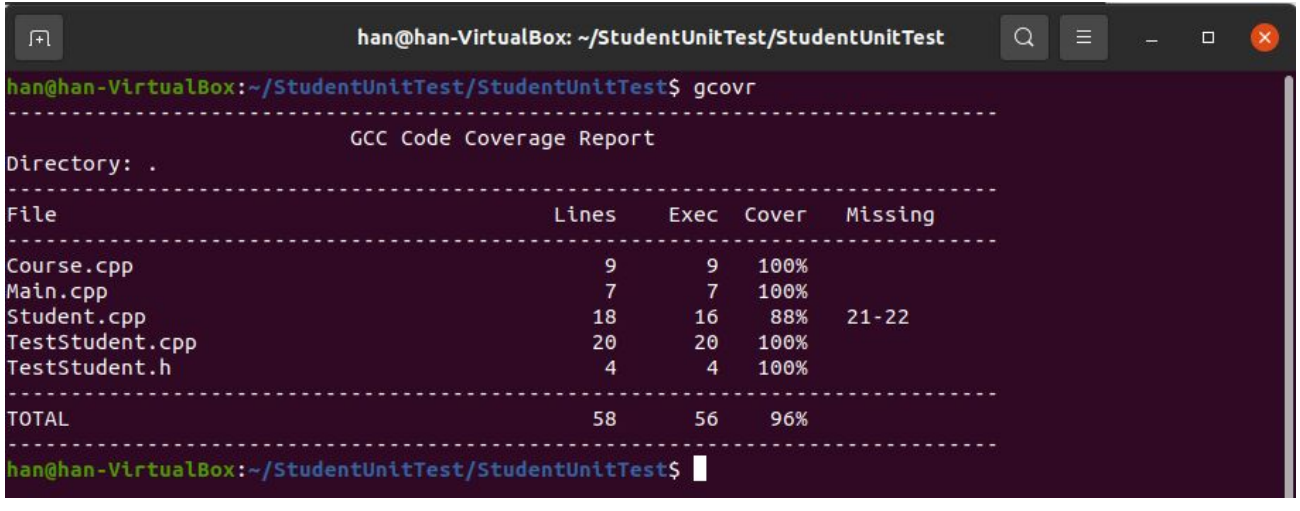
Compile & Run

- $ g++ --coverage -g -O0 -o output_filename [cppfiles] -lcppunit
  - coverage file(.gcno) is generated after compiling
- $ ./output_filename
  - run the executable file then the coverage data(.gcda) is generated



Check Line Coverage

- $ gcovr
  - analyzing coverage data generated when compiling and running



Requirement of Project1

- Find an external library and do unit test on it with CppUnit
- Write a report specifies
  - testcases you designed
  - testing result
  - external library you found

Scoring

- rawScore(you) = #of bugs-found + line-coverage * log2(# of total lines of your library SUT)
  - # of bugs-found is at most 3
- projectScore(you) = 15 * rawScore(you) / max(rawScore(anyStudents))

Submission

- Hand in the following files to NTU COOL
  - the external library you found
  - your testing code
  - report
- Deadline: 2023/4/12 23:59